

Chapter 30 — Minimum Spanning Tree

0. What You Will Learn in This Chapter

A Minimum Spanning Tree, or MST, is a graph pattern for connecting all nodes with minimum total edge cost.

It applies to:

undirected weighted connected graphs

A spanning tree connects all nodes using exactly:

$n - 1$ edges

A minimum spanning tree is the spanning tree with the smallest total weight.

By the end of this chapter, you should understand:

- What a spanning tree is
- What a minimum spanning tree is
- When MST applies
- How MST differs from shortest path
- How Kruskal's algorithm works
- How Union Find supports Kruskal
- How Prim's algorithm works
- When to use Kruskal vs Prim
- How to solve Min Cost to Connect All Points
- How to solve Connecting Cities With Minimum Cost
- How to solve Critical and Pseudo-Critical Edges
- How to recognize MST problems
- Common MST mistakes

Core problems covered in this chapter:

1. Min Cost to Connect All Points
2. Connecting Cities With Minimum Cost
3. Optimize Water Distribution in a Village
4. Find Critical and Pseudo-Critical Edges in Minimum Spanning Tree
5. Minimum Cost to Connect Sticks, contrast
6. Network Delay Time, contrast with shortest path
7. Path With Minimum Effort, MST connection
8. Swim in Rising Water, MST/Union Find connection
9. Redundant Connection, contrast
10. Kruskal template
11. Prim template
12. MST with virtual node
13. MST edge classification
14. Minimum Bottleneck Spanning Tree concept

1. What Is a Spanning Tree?

A spanning tree is a subset of edges that:

15. connects all nodes
16. has no cycles

17. uses exactly $n - 1$ edges for n nodes

Example graph:

```
0 --1-- 1
|      |
4      2
|      |
2 --3-- 3
```

A spanning tree might use edges:

0-1
1-3
0-2

It connects all nodes:

0, 1, 2, 3

and has no cycle.

2. What Is a Minimum Spanning Tree?

A minimum spanning tree is the spanning tree with minimum total edge weight.

Example:

```
0 --1-- 1
|      |
4      2
|      |
2 --3-- 3
```

Possible spanning tree:

0-1 cost 1
1-3 cost 2
2-3 cost 3

Total:

6

Another spanning tree:

0-2 cost 4
2-3 cost 3
1-3 cost 2

Total:

9

The first one is better.

MST asks for the minimum total cost to connect all nodes.

3. MST vs Shortest Path

This is extremely important.

minimum cost from one node to another	shortest path
minimum cost from one node to all nodes	shortest path tree
minimum total cost to connect all nodes	MST
can every node receive signal as fast as possible from source	shortest path
build roads/cables to connect all cities cheaply	MST

Example:

```
      100
0 ----- 1
|         |
1         1
|         |
2 ----- 3
      1
```

Shortest path from 0 to 1 might be direct cost 100 or indirect cost 3.

MST instead wants to connect all nodes with minimum total edge cost.

It may include edges unrelated to a specific source-target shortest path.

4. Core Need / Why Use MST?

Use MST when the problem asks:

Connect all nodes with minimum total cost.

Examples:

- connect cities with roads
- connect computers with cables
- connect points with Manhattan distance
- provide water to all houses with wells/pipes
- connect islands with bridges
- choose edges so the whole graph becomes connected as cheaply as possible

MST is not about reaching a target quickly.

It is about building a minimum-cost connected network.

5. When to Use MST

Use MST when:

5.1 Need Connect All Nodes

Example:

Connect all cities.

5.2 Edges Have Costs

Example:

Road from city A to B costs 5.

5.3 Need Minimum Total Cost

Example:

Minimize total construction cost.

5.4 Graph Is Undirected

MST is normally for undirected graphs.

5.5 Need No Cycles

A cycle is wasteful for connecting all nodes because one edge in a cycle can be removed while still staying connected.

6. When NOT to Use MST

6.1 Need Shortest Path From Source

Use Dijkstra/BFS/Bellman-Ford.

Example:

- Network Delay Time
- Cheapest Flights
- Path With Maximum Probability

6.2 Directed Graph Dependencies

Use topological sort or directed graph algorithms.

6.3 Need Dynamic Connectivity Only

Use Union Find.

6.4 Need Maximum Flow

Use max-flow algorithms.

6.5 Need Hamiltonian Path / Visit All Nodes Once

MST is not enough.

7. MST Properties

7.1 MST Has $n - 1$ Edges

For n nodes, any spanning tree has:

$n - 1$ edges

If you use fewer, graph is disconnected.

If you use more, graph has a cycle.

7.2 MST May Not Be Unique

If multiple edges have same weights, more than one MST can exist.

Example:

triangle with all edges weight 1

Any two edges form an MST.

7.3 Cycles Are Avoided

When building MST, if an edge connects two nodes already connected, adding it creates a cycle.

Skip it.

7.4 Cut Property

For any cut dividing nodes into two groups, the lightest edge crossing the cut is safe to include in some MST.

This is the idea behind Prim and Kruskal.

8. Two Main MST Algorithms

There are two main MST algorithms:

18. Kruskal's algorithm
 19. Prim's algorithm
-

9. Kruskal's Algorithm

Kruskal's algorithm sorts all edges by weight.

Then it adds edges from smallest to largest, skipping edges that create cycles.

Steps:

20. Put all edges in a list.
 21. Sort edges by weight.
 22. Initialize Union Find.
 23. For each edge from smallest to largest:
 - if endpoints are in different components, add edge
 - union endpoints
 - add cost
 24. Stop when $n - 1$ edges are used.
-

10. Kruskal Mental Model

Imagine building roads.

Always choose the cheapest remaining road that connects two currently separate regions.

If a road connects cities already connected through other roads, it is unnecessary and creates a cycle.

Skip it.

11. Kruskal Template

```

class DSU {
public:
    vector<int> parent;
    vector<int> size;

    DSU(int n) {
        parent.resize(n);
        size.resize(n, 1);

        for (int i = 0; i < n; i++) {
            parent[i] = i;
        }
    }

    int find(int x) {
        if (parent[x] != x) {
            parent[x] = find(parent[x]);
        }

        return parent[x];
    }

    bool unite(int a, int b) {
        int rootA = find(a);
        int rootB = find(b);

        if (rootA == rootB) {
            return false;
        }

        if (size[rootA] < size[rootB]) {
            swap(rootA, rootB);
        }

        parent[rootB] = rootA;
        size[rootA] += size[rootB];

        return true;
    }
};

int kruskal(int n, vector<vector<int>>& edges) {

```

```

sort(edges.begin(), edges.end());

DSU dsu(n);

int totalCost = 0;
int usedEdges = 0;

for (auto& edge : edges) {
    int weight = edge[0];
    int u = edge[1];
    int v = edge[2];

    if (dsu.unite(u, v)) {
        totalCost += weight;
        usedEdges++;

        if (usedEdges == n - 1) {
            break;
        }
    }
}

if (usedEdges != n - 1) {
    return -1;
}

return totalCost;
}

```

This template assumes edges are stored as:

```
{weight, u, v}
```

12. Prim's Algorithm

Prim's algorithm grows MST from one starting node.

Steps:

25. Start from any node.
26. Push all outgoing edges into min-heap.
27. Always choose cheapest edge that reaches an unvisited node.
28. Add that node to MST.
29. Push its outgoing edges.
30. Continue until all nodes are visited.

13. Prim Mental Model

Imagine one connected island of nodes.

At every step, choose the cheapest bridge from the current island to a new node outside the island.

This grows the MST one node at a time.

14. Prim Template

```
int prim(int n, vector<vector<pair<int, int>>>& graph) {
    vector<bool> visited(n, false);

    priority_queue<pair<int, int>,
        vector<pair<int, int>>,
        greater<pair<int, int>>> pq;

    pq.push({0, 0});

    int totalCost = 0;
    int visitedCount = 0;

    while (!pq.empty() && visitedCount < n) {
        auto [cost, node] = pq.top();
        pq.pop();

        if (visited[node]) {
            continue;
        }

        visited[node] = true;
        visitedCount++;
        totalCost += cost;

        for (auto [neighbor, weight] : graph[node]) {
            if (!visited[neighbor]) {
                pq.push({weight, neighbor});
            }
        }
    }

    if (visitedCount != n) {
        return -1;
    }
}
```

```

    return totalCost;
}

```

15. Kruskal vs Prim

Main structure	edge list + Union Find	adjacency list + min-heap
Sort all edges?	yes	no
Good when	edges list is natural	adjacency list is natural
Sparse graph	good	good
Dense graph	sorting many edges costly	can be good with optimized matrix
Cycle prevention	Union Find	visited set
Stops when	n - 1 edges used	all nodes visited

Use Kruskal when the input is an edge list.

Use Prim when the graph is already in adjacency list form or edges are generated from current frontier.

16. Major MST Variations

Basic MST	connect all nodes min cost	Connecting Cities
Complete graph generated from points	generate all pair edges	Min Cost Points
Virtual node	choose between node cost and edge cost	Water Distribution
Critical edges	compare MST with/without edge	Critical Edges
Bottleneck path	MST can answer min max-edge path	Path With Minimum Effort concept
Dynamic threshold connectivity	sort edges and union	offline connectivity

Variation	Core Idea	Example
Maximum spanning tree	sort descending	max total connection

17. Full Problem Walkthrough 1 — LeetCode 1584: Min Cost to Connect All Points

17.1 Problem Summary

Given points on 2D plane.

Cost to connect two points is Manhattan distance:

$$\text{abs}(x1 - x2) + \text{abs}(y1 - y2)$$

Return minimum cost to connect all points.

17.2 How to Identify MST

Need connect all points.

Every pair of points can be connected.

Need minimum total connection cost.

This is MST on a complete graph.

17.3 Kruskal Approach

Build all pair edges:

$$\text{cost}, i, j$$

Sort by cost.

Use Union Find.

17.4 C++ Code — Kruskal

```
class Solution {
public:
    class DSU {
    public:
        vector<int> parent;
        vector<int> size;

        DSU(int n) {
            parent.resize(n);
            size.resize(n, 1);

            for (int i = 0; i < n; i++) {
                parent[i] = i;
            }
        }
    };
};
```

```

    }
}

int find(int x) {
    if (parent[x] != x) {
        parent[x] = find(parent[x]);
    }

    return parent[x];
}

bool unite(int a, int b) {
    int rootA = find(a);
    int rootB = find(b);

    if (rootA == rootB) {
        return false;
    }

    if (size[rootA] < size[rootB]) {
        swap(rootA, rootB);
    }

    parent[rootB] = rootA;
    size[rootA] += size[rootB];

    return true;
}

};

int minCostConnectPoints(vector<vector<int>>& points) {
    int n = points.size();

    vector<vector<int>> edges;

    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            int cost = abs(points[i][0] - points[j][0]) +
                abs(points[i][1] - points[j][1]);

            edges.push_back({cost, i, j});
        }
    }
}

```

```

    }

    sort(edges.begin(), edges.end());

    DSU dsu(n);

    int totalCost = 0;
    int usedEdges = 0;

    for (auto& edge : edges) {
        int cost = edge[0];
        int u = edge[1];
        int v = edge[2];

        if (dsu.unite(u, v)) {
            totalCost += cost;
            usedEdges++;

            if (usedEdges == n - 1) {
                break;
            }
        }
    }

    return totalCost;
}
};

```

17.5 Complexity

There are:

$O(n^2)$

edges.

Time:

$O(n^2 \log n)$

Space:

$O(n^2)$

17.6 Prim Alternative

Prim can avoid explicitly storing all edges.

For complete graph, maintain best cost to connect each unvisited point.

17.7 C++ Code — Prim $O(n^2)$

```
class Solution {
public:
    int minCostConnectPoints(vector<vector<int>>& points) {
        int n = points.size();

        vector<bool> visited(n, false);
        vector<int> minDist(n, INT_MAX);

        minDist[0] = 0;

        int totalCost = 0;

        for (int step = 0; step < n; step++) {
            int node = -1;

            for (int i = 0; i < n; i++) {
                if (!visited[i] &&
                    (node == -1 || minDist[i] < minDist[node])) {
                    node = i;
                }
            }

            visited[node] = true;
            totalCost += minDist[node];

            for (int next = 0; next < n; next++) {
                if (visited[next]) {
                    continue;
                }

                int cost = abs(points[node][0] - points[next][0]) +
                    abs(points[node][1] - points[next][1]);

                minDist[next] = min(minDist[next], cost);
            }
        }

        return totalCost;
    }
};
```

17.8 Complexity of Prim Version

Time:

$O(n^2)$

Space:

$O(n)$

This is often better for complete graph point problems.

18. Full Problem Walkthrough 2 — Connecting Cities With Minimum Cost

18.1 Problem Summary

Given n cities labeled $1..n$ and connections:

city1, city2, cost

Return minimum cost to connect all cities.

If impossible, return -1.

18.2 How to Identify MST

Need connect all cities with minimum total cost.

Edges are undirected weighted connections.

Use Kruskal.

18.3 C++ Code

```
int minimumCost(int n, vector<vector<int>>& connections) {
    sort(connections.begin(),
          connections.end(),
          [](const vector<int>& a, const vector<int>& b) {
              return a[2] < b[2];
          });

    DSU dsu(n + 1);

    int totalCost = 0;
    int usedEdges = 0;

    for (auto& edge : connections) {
        int u = edge[0];
        int v = edge[1];
        int cost = edge[2];
```

```
    if (dsu.unite(u, v)) {
        totalCost += cost;
        usedEdges++;

        if (usedEdges == n - 1) {
            break;
        }
    }
}

return usedEdges == n - 1 ? totalCost : -1;
}
```

18.4 Complexity

Time:

$O(E \log E)$

Space:

$O(n)$

18.5 Common Mistake

Cities are 1-indexed.

Use DSU size $n + 1$.

19. Full Problem Walkthrough 3 — Optimize Water Distribution in a Village

19.1 Problem Summary

There are n houses.

Each house can either:

- build its own well with cost `wells[i]`
- connect to another house by pipe with cost

Need minimum cost to supply water to every house.

19.2 How to Identify MST With Virtual Node

Each house can get water from a well or from pipes.

Create virtual node 0.

Building a well at house i becomes edge:

```
0 -- i cost wells[i - 1]
```

Pipes are normal edges.
Now the problem becomes:

connect all houses and virtual water source with minimum cost

Use MST.

19.3 C++ Code

```
class Solution {
public:
    int minCostToSupplyWater(int n,
                             vector<int>& wells,
                             vector<vector<int>>& pipes) {
        vector<vector<int>> edges;

        for (int house = 1; house <= n; house++) {
            edges.push_back({wells[house - 1], 0, house});
        }

        for (auto& pipe : pipes) {
            int u = pipe[0];
            int v = pipe[1];
            int cost = pipe[2];

            edges.push_back({cost, u, v});
        }

        sort(edges.begin(), edges.end());

        DSU dsu(n + 1);

        int totalCost = 0;

        for (auto& edge : edges) {
            int cost = edge[0];
            int u = edge[1];
            int v = edge[2];

            if (dsu.unite(u, v)) {
                totalCost += cost;
            }
        }
    }
};
```

```
        return totalCost;
    }
};
```

19.4 Complexity

Let:

$E = \text{pipes.size()} + n$

Time:

$O(E \log E)$

Space:

$O(E)$

19.5 Key Lesson

When every node has an individual activation cost, consider adding a virtual source node.

20. Full Problem Walkthrough 4 — LeetCode 1489: Critical and Pseudo-Critical Edges in MST

20.1 Problem Summary

Given weighted undirected graph.

An edge is **critical** if removing it increases MST cost or makes MST impossible.

An edge is **pseudo-critical** if it can appear in some MST but is not critical.

Return lists of critical and pseudo-critical edge indices.

20.2 How to Identify MST Edge Classification

Need reason about MST membership.

Use Kruskal repeatedly:

31. Compute original MST cost.
32. For each edge:
 - exclude it and compute MST
 - if cost increases or impossible, edge is critical
 - otherwise force include it and compute MST
 - if cost equals original MST cost, edge is pseudo-critical

20.3 Add Original Index

Sort edges by weight, but preserve original index.

Edge format:

```
{weight, u, v, index}
```

20.4 Kruskal Helper

```
int buildMST(int n,  
            vector<vector<int>>& edges,  
            int skipIndex,  
            int forceIndex) {  
    DSU dsu(n);  
  
    int cost = 0;  
    int used = 0;  
  
    if (forceIndex != -1) {  
        auto& edge = edges[forceIndex];  
  
        int w = edge[0];  
        int u = edge[1];  
        int v = edge[2];  
  
        if (dsu.unite(u, v)) {  
            cost += w;  
            used++;  
        }  
    }  
  
    for (int i = 0; i < edges.size(); i++) {  
        if (i == skipIndex) {  
            continue;  
        }  
  
        auto& edge = edges[i];  
  
        int w = edge[0];  
        int u = edge[1];  
        int v = edge[2];  
  
        if (dsu.unite(u, v)) {  
            cost += w;  
            used++;  
  
            if (used == n - 1) {
```

```

        break;
    }
}

if (used != n - 1) {
    return INT_MAX;
}

return cost;
}

```

20.5 Full C++ Code

```

class Solution {
public:
    class DSU {
    public:
        vector<int> parent;
        vector<int> size;

        DSU(int n) {
            parent.resize(n);
            size.resize(n, 1);

            for (int i = 0; i < n; i++) {
                parent[i] = i;
            }
        }

        int find(int x) {
            if (parent[x] != x) {
                parent[x] = find(parent[x]);
            }

            return parent[x];
        }

        bool unite(int a, int b) {
            int rootA = find(a);
            int rootB = find(b);

            if (rootA == rootB) {

```

```

        return false;
    }

    if (size[rootA] < size[rootB]) {
        swap(rootA, rootB);
    }

    parent[rootB] = rootA;
    size[rootA] += size[rootB];

    return true;
}
};

int buildMST(int n,
             vector<vector<int>>& edges,
             int skipIndex,
             int forceIndex) {
    DSU dsu(n);

    int cost = 0;
    int used = 0;

    if (forceIndex != -1) {
        auto& edge = edges[forceIndex];

        int w = edge[0];
        int u = edge[1];
        int v = edge[2];

        if (dsu.unite(u, v)) {
            cost += w;
            used++;
        }
    }

    for (int i = 0; i < edges.size(); i++) {
        if (i == skipIndex) {
            continue;
        }

        auto& edge = edges[i];

```

```

    int w = edge[0];
    int u = edge[1];
    int v = edge[2];

    if (dsu.unite(u, v)) {
        cost += w;
        used++;

        if (used == n - 1) {
            break;
        }
    }
}

if (used != n - 1) {
    return INT_MAX;
}

return cost;
}

vector<vector<int>> findCriticalAndPseudoCriticalEdges(
    int n,
    vector<vector<int>>& inputEdges
){
    vector<vector<int>> edges;

    for (int i = 0; i < inputEdges.size(); i++) {
        int u = inputEdges[i][0];
        int v = inputEdges[i][1];
        int w = inputEdges[i][2];

        edges.push_back({w, u, v, i});
    }

    sort(edges.begin(), edges.end());

    int baseCost = buildMST(n, edges, -1, -1);

    vector<int> critical;
    vector<int> pseudoCritical;

```

```

for (int i = 0; i < edges.size(); i++) {
    int withoutCost = buildMST(n, edges, i, -1);

    if (withoutCost > baseCost) {
        critical.push_back(edges[i][3]);
        continue;
    }

    int withCost = buildMST(n, edges, -1, i);

    if (withCost == baseCost) {
        pseudoCritical.push_back(edges[i][3]);
    }
}

return {critical, pseudoCritical};
}
};

```

20.6 Complexity

For each edge, Kruskal may run twice.

Time:

$O(E^2 * \alpha(V))$

plus sorting:

$O(E \log E)$

This is acceptable for typical constraints of this problem.

21. Full Problem Walkthrough 5 — Minimum Cost to Connect Sticks, Contrast

21.1 Problem Summary

Given stick lengths.

Combine two sticks at cost equal to their sum.

Return minimum total cost to combine all sticks.

21.2 Why This Is Not MST

There are no graph nodes and edges to connect.

This is a greedy heap problem like Huffman coding.

Use min-heap:

33. pop two smallest
 34. combine
 35. push sum
-

21.3 C++ Code

```
int connectSticks(vector<int>& sticks) {
    priority_queue<int, vector<int>, greater<int>> pq;

    for (int x : sticks) {
        pq.push(x);
    }

    int total = 0;

    while (pq.size() > 1) {
        int a = pq.top();
        pq.pop();

        int b = pq.top();
        pq.pop();

        int cost = a + b;
        total += cost;

        pq.push(cost);
    }

    return total;
}
```

21.4 Key Lesson

Not every “connect with minimum cost” problem is MST.
MST requires graph nodes and selectable edges.

22. Full Problem Walkthrough 6 — Network Delay Time, Contrast

Network Delay Time asks:

How long until all nodes receive signal from source?

This is shortest path from one source to all nodes.

Use Dijkstra, not MST.

Why?

MST minimizes total edge construction cost.

Network delay minimizes maximum shortest-path arrival time from a source.

These are different goals.

23. Full Problem Walkthrough 7 — Path With Minimum Effort, MST Connection

Path With Minimum Effort asks for path from top-left to bottom-right minimizing maximum edge difference.

This can be solved by Dijkstra.

It also has an MST interpretation:

If you sort all grid edges by effort and union them from smallest to largest, the first time start and target become connected, that effort is the answer.

Why?

MST preserves minimax path property.

23.1 Union Find Version Sketch

```
sort(edges.begin(), edges.end());

DSU dsu(rows * cols);

for (auto& edge : edges) {
    int effort = edge[0];
    int a = edge[1];
    int b = edge[2];

    dsu.unite(a, b);

    if (dsu.connected(0, rows * cols - 1)) {
        return effort;
    }
}
```

24. Full Problem Walkthrough 8 — Swim in Rising Water, MST Connection

Swim in Rising Water can also be seen as connectivity under threshold.
Sort cells or edges by height/time.
As time rises, more cells become available.
Use Union Find to connect available neighbors.
First time top-left and bottom-right are connected, that time is answer.
This is not a classic “return MST total cost” problem, but it uses MST-like threshold connectivity.

25. Common Edge Cases for MST

25.1 Graph Not Connected

No spanning tree exists.
Return -1 or handle according to problem.
Kruskal check:

```
usedEdges == n - 1
```

Prim check:

```
visitedCount == n
```

25.2 Single Node

MST cost is usually 0.
No edges needed.

25.3 Duplicate Edge Weights

MST may not be unique.
Do not assume a single MST.

25.4 Multiple Edges Between Same Nodes

Keep all edges.
Kruskal naturally chooses cheaper useful one.

25.5 Negative Edge Weights

MST algorithms still work with negative edge weights.
Unlike Dijkstra, MST does not require non-negative weights.

25.6 1-Indexed Nodes

Use DSU size $n + 1$.

25.7 Complete Graph

Edges may be $O(n^2)$.
Consider Prim $O(n^2)$ to save memory.

26. Common Mistakes and Wrong Code

Mistake 1 — Using Dijkstra for MST

Dijkstra solves shortest paths from a source.
MST solves minimum total cost to connect all nodes.

Mistake 2 — Forgetting Cycle Check in Kruskal

Wrong:

```
total += cost;
```

for every sorted edge.

Correct:

```
if (dsu.unite(u, v)) {  
    total += cost;  
}
```

Mistake 3 — Not Checking Connectivity

If graph is disconnected, Kruskal may use fewer than $n - 1$ edges.

Need:

```
if (usedEdges != n - 1) {  
    return -1;  
}
```

Mistake 4 — Sorting Wrong Field

If edge is $\{u, v, \text{cost}\}$, sort by cost, not by u .

Either use comparator or store as $\{\text{cost}, u, v\}$.

Mistake 5 — Wrong Indexing

Cities often use $1..n$.

Points often use $0..n-1$.

Mistake 6 — Forgetting Virtual Node

In water distribution, building a well is not separate from MST.

Model it as an edge from virtual node.

27. Interview Explanation Templates

27.1 MST Recognition

The problem asks for the minimum total cost to connect all nodes, so I model it as a minimum spanning tree problem on an undirected weighted graph.

27.2 Kruskal

I sort all edges by cost. Then I iterate from cheapest to most expensive and add an edge only if it connects two different components. Union Find lets me detect whether adding the edge would create a cycle.

27.3 Prim

I grow the MST from an arbitrary starting node. I use a min-heap of candidate edges crossing from the visited set to unvisited nodes, always choosing the cheapest next edge.

27.4 Virtual Node

For each individual build option, I add an edge from a virtual source node to that item. Then choosing between building directly and connecting through edges becomes a normal MST problem.

27.5 MST vs Shortest Path

Shortest path minimizes distance from a source to a target or all nodes. MST minimizes total edge cost to connect every node. Since this problem asks to connect all nodes with minimum total cost, MST is the right model.

28. Complexity Patterns

Kruskal	$O(E \log E)$	$O(V)$ plus edges
Prim with heap	$O(E \log E)$	$O(V + E)$
Prim dense $O(n^2)$	$O(V^2)$	$O(V)$
Min Cost Connect Points, Kruskal	$O(n^2 \log n)$	$O(n^2)$
Min Cost Connect Points, Prim	$O(n^2)$	$O(n)$
Connecting Cities	$O(E \log E)$	$O(V)$
Water Distribution	$O((E + V) \log(E + V))$	$O(E + V)$
Critical Edges	$O(E^2 \alpha(V))$	$O(V + E)$

29. Time-Boxed Practice Guidance

Intro MST

Target:

20–35 minutes

Examples:

- Connecting Cities With Minimum Cost
- Min Cost to Connect All Points, Kruskal
- Min Cost to Connect All Points, Prim

Medium MST

Target:

35–55 minutes

Examples:

- Optimize Water Distribution
- Path With Minimum Effort, Union Find version
- Swim in Rising Water, Union Find version

Harder MST

Target:

60–90 minutes

Examples:

- Critical and Pseudo-Critical Edges
- Find Critical and Pseudo-Critical Edges in MST
- Checking Existence of Edge Length Limited Paths, offline Union Find contrast

30. Expanded Practice Set

30.1 Must-Do Medium

36. Min Cost to Connect All Points
37. Connecting Cities With Minimum Cost
38. Optimize Water Distribution in a Village
39. Path With Minimum Effort, Union Find version
40. Swim in Rising Water, Union Find version

30.2 Strong Interview Practice

41. Find Critical and Pseudo-Critical Edges in Minimum Spanning Tree
42. Minimum Cost to Connect All Points, Prim $O(n^2)$
43. Connecting Cities With Minimum Cost, Kruskal
44. Checking Existence of Edge Length Limited Paths
45. Remove Max Number of Edges to Keep Graph Fully Traversable

30.3 Hard / Stretch

46. Critical and Pseudo-Critical Edges
47. Minimize Malware Spread
48. Bricks Falling When Hit, reverse Union Find
49. Last Day Where You Can Still Cross
50. Find Critical and Pseudo-Critical Edges
51. Minimum Cost to Connect Two Groups of Points, note: DP not MST

31. Suggested Study Order

Step 1 — Kruskal Basics

Start with:

52. Implement DSU
53. Kruskal template
54. Connecting Cities With Minimum Cost

Goal:

Understand sorting edges and skipping cycles.

Step 2 — Complete Graph MST

Solve:

55. Min Cost to Connect All Points with Kruskal
56. Min Cost to Connect All Points with Prim

Goal:

Understand edge generation and memory tradeoffs.

Step 3 — Virtual Node MST

Solve:

57. Optimize Water Distribution

Goal:

Model node activation costs as edges.

Step 4 — MST vs Shortest Path

Compare:

58. Network Delay Time
59. Min Cost Connect Points
60. Path With Minimum Effort

Goal:

Avoid confusing shortest path and MST.

Step 5 — Advanced MST

Solve:

61. Critical and Pseudo-Critical Edges

Goal:

Classify edges by MST necessity.

32. Pattern Decision Table

connect all nodes minimum total cost	MST
edge list given	Kruskal
need avoid cycles	Union Find
graph already adjacency list	Prim
complete graph from points	Prim $O(n^2)$ or Kruskal
each node can be built independently	virtual node
edge necessary in every MST	critical edge check
edge can appear in some MST	force edge and compare cost
minimize maximum edge on path	MST/Union Find threshold possible
shortest from source	not MST

33. Final Cheat Sheet

Kruskal

```
sort(edges.begin(), edges.end());
```

```
DSU dsu(n);
```

```
int cost = 0;
```

```
int used = 0;
```

```
for (auto& edge : edges) {
```

```
    int w = edge[0];
```

```
    int u = edge[1];
```

```
    int v = edge[2];
```

```
    if (dsu.unite(u, v)) {
```

```
        cost += w;
```

```
        used++;
```

```
    }
```

```
}
```

```
if (used != n - 1) {
```

```
    return -1;
```

```
}
```

```
return cost;
```

Prim

```
priority_queue<pair<int, int>,  
              vector<pair<int, int>>,  
              greater<pair<int, int>>> pq;
```

```
pq.push({0, 0});
```

```
while (!pq.empty()) {
```

```
    auto [cost, node] = pq.top();
```

```
    pq.pop();
```

```
    if (visited[node]) {
```

```
        continue;
```

```
    }
```

```
    visited[node] = true;
```

```
    total += cost;
```

```

for (auto [nei, w] : graph[node]) {
    if (!visited[nei]) {
        pq.push({w, nei});
    }
}
}

```

Virtual Node

```

for (int i = 1; i <= n; i++) {
    edges.push_back({wellCost[i], 0, i});
}

```

Critical Edge Test

If MST without edge > base MST cost, edge is critical.

Pseudo-Critical Edge Test

If MST with edge forced == base MST cost, edge is pseudo-critical.

34. What to Master Before Chapter 31

Before moving to Backtracking, make sure you can:

- Explain what MST means
- Distinguish MST from shortest path
- Implement Kruskal
- Use Union Find in Kruskal
- Know why Kruskal skips cycle edges
- Implement Prim with min-heap
- Choose Kruskal vs Prim
- Handle disconnected graphs
- Handle 1-indexed nodes
- Solve Min Cost to Connect All Points
- Solve Connecting Cities With Minimum Cost
- Use virtual node modeling
- Understand critical and pseudo-critical edge classification
- Explain MST complexity

35. Core Takeaway

Minimum Spanning Tree is for connecting all nodes with minimum total edge cost.
The key phrase is:

connect everything as cheaply as possible

Use:

- **Kruskal** when you have an edge list

- **Prim** when you grow from a node using adjacency/frontier edges
- **Union Find** to detect cycles in Kruskal
- **virtual node** when each node has its own build/activation cost

Always separate MST from shortest path:

shortest path = cheapest route from source
MST = cheapest network connecting all nodes

If the problem asks for minimum total cost to connect all nodes in an undirected weighted graph, think MST first.